Activity 5 Report: Database Maintenance and Monitoring

COMP 018 Database Administration | Project tested: BoardTrack (Boarding House and Rental Management) (Node.js + PostgreSQL) | Date of testing: 6 November 2026

1. Cover and links

Item	Value
Testing group	G08: Borreta, Joshua; Bola, Martin Lawrence; Juatchon, Joaquin Phillip; Marcos, Karl Raphael; Pacheco Jr., Sancho; Vera Cruz, Euan Joseph
Project tested	BoardTrack (Boarding House and Rental Management)
Owning group	Group 11 (BoardTrack)
Repository	https://github.com/karlrobertc-art/BoardTrack
Tracker	https://github.com/EclipseChan/bwisIT-Solutions---Campus-Canteen-Pre-Order-and-Queue-Management-System/tree/main/other_group_tests
Evidence files	a5_health.txt, a5_logs.txt, a5_maintenance.txt

Run on 6 November 2026 at 23:44 (Asia/Manila), after the application tests of Activities 3 and 4. Results come from the saved files.

2. Health and activity

Check	Result
Version	PostgreSQL 18.6, x86_64 Windows
Database size	9070 kB
Connections	1 backend (the monitoring session). max_connections is 100
Transactions	177 commits, 11 rollbacks (from the constraint and permission tests)
Cache	66,139 buffer hits against 16 reads, about 99.98% hit ratio
Deadlocks and temp files	0 and 0
Autovacuum	on. No table had been vacuumed or analyzed before this run
Rows	app_user 3, lease 4, payment 7, property 2, room 6, schema_migrations 1, tenant 4, utility_bill 4

Dead tuples before maintenance: room 7, lease 6, property 2, tenant 1. They come from the updates made by the trigger and the tests and are below the autovacuum thresholds.

3. Logging

Setting	Value	Comment
log_statement	none	No statement logging
log_min_duration_statement	-1	Slow queries are not logged. Recommend 500 ms
Log folder	C:/Program Files/PostgreSQL/18/data/log	Readable

4. Log findings

Time	Entry	Explanation
23:34 to 23:37	FATAL: database "boardtrack_db" does not exist (about 28 times)	The tester's own checks ran while the app could not start, because the first two attempts of the script did not install the packages. Not an application fault
23:38:46 to 23:38:47	Exclusion violation, room status guard message, foreign key violation, check violations, invalid date, duplicate email	The deliberate rule tests of Activity 3. The app log shows the same codes (23P01, P0001, 23503, 23514, 22007, 23505)
23:41:43 to 23:42:14	Exclusion violation (3 times)	Overlapping lease attempts made in the browser for the screenshot
23:43:34 to 23:43:35	permission denied for table lease, payment, property; for schema public; must be owner; to create role	The access tests of Activity 4 on the copy database
23:44:12	permission denied to vacuum "lease"	A non-owner role trying maintenance (see section 5)
Earlier in the day	Errors and failed logins from other projects tested on the same server	Not part of this project

No unexplained errors from the application. The Node log shows only the expected database error codes, one line each.

5. Maintenance performed

Action	Result
ANALYZE (all tables)	Completed. last_analyze set on all 8 tables
VACUUM (VERBOSE, ANALYZE) public.lease	6 dead row versions removed, 4 live rows remain, n_dead_tup for lease went from 6 to 0
VACUUM as a non-owner role (boardtrack_app from the Activity 4 copy)	Skipped with "permission denied to vacuum". The project defines no maintenance role

6. Query plan

Query: the Leases list for one tenant (v_lease_summary filtered by tenant_id = 1), the query the tenant screen runs. EXPLAIN (ANALYZE, BUFFERS) showed nested loops over sequential scans, planning time 8.35 ms and execution time 0.14 ms, 1 row returned. On tables of 2 to 7 rows a sequential scan is the right plan.

The database has 16 indexes. The foreign keys on lease and the child tables are indexed (idx_lease_room, idx_lease_tenant, idx_payment_lease, idx_utility_lease). Two foreign key columns have none: room.property_id and app_user.tenant_id.

7. Recommendations

- Create a maintenance role with pg_maintain and use it for scheduled VACUUM and ANALYZE.
- Set log_min_duration_statement to a value such as 500 ms and keep log_statement at ddl.
- Add indexes on room(property_id) and app_user(tenant_id) before the data grows.

- Re-run EXPLAIN on the lease summary when there are thousands of payments, because the view totals them per lease.
- Monitor dead tuples (the status updates by the trigger create them) and give a monitoring role pg_monitor.

8. Feedback sent to owners

Sent on 6 November 2026: no health problems found. Recommended a maintenance role, slow-query logging and two foreign key indexes.

AI usage declaration

- Tool: Claude (Anthropic).
- Tasks: interpreting the results of the testing.
- Validation: the commands were run by the group on its own server, and every result in this report comes from the saved output files.